

MASTER PROJECT DOCUMENTATION

Diabetes Risk Prediction

A Machine Learning Pre-Screening Tool — Technical Report, Portfolio Case Study & Career Toolkit

Field	Detail
Author	Yemi Fatodu
Project Category	Data Science / Machine Learning — Healthcare
Status	Completed — model pipeline; live deployment
Project Duration	[Information Needed]
Repository	github.com/yemifatodu/diabetes-prediction-app
Live Application	[Insert Streamlit URL]
Portfolio	yemifatodu.online

This document consolidates eight deliverables — Executive Summary, Complete Technical Report, Portfolio Case Study, Portfolio Website Copy, Interview Preparation, Recruiter Summary, Resume Bullet Points, and Portfolio SEO Content — into a single reference package. Every factual claim is drawn from the project's notebooks, documentation, and results. Fields without confirmed source data are marked [Information Needed] rather than invented.

1. Executive Summary

Diabetes affects over 537 million adults worldwide, and early detection materially improves outcomes — but full clinical workups are resource-intensive and not universally accessible. This project builds and deploys a machine learning model that flags high diabetes risk from eight routine clinical measurements, offering a low-cost pre-screening signal that could help prioritize patients for follow-up testing.

What was built

- A full data science pipeline: data cleaning, exploratory analysis, feature engineering, model training, and evaluation on the Pima Indians Diabetes dataset (768 patient records, NIDDK).
- Three candidate models — Logistic Regression, Random Forest, and XGBoost — trained and compared using 5-fold cross-validation.
- A production-facing Streamlit web application that lets a user enter clinical values and receive a live risk prediction.
- Public deployment: code on GitHub, application hosted on Streamlit Cloud, and the project featured on a personal portfolio site.

Headline results

Metric	Result
Best model	XGBoost
Test accuracy	88.31% (vs. 65.1% naive baseline)
AUC-ROC	0.947
Precision (diabetic class)	82.14%
Recall (diabetic class)	85.19%

Why it matters

The project demonstrates the full lifecycle expected of a production data science deliverable: catching a subtle, high-impact data quality issue (biologically impossible zero values hidden in five clinical columns), addressing class imbalance honestly rather than optimizing for a misleading accuracy figure, and shipping a working, publicly accessible application rather than stopping at a notebook.

2. Complete Technical Report

2.1 Introduction

This report documents the design, implementation, and deployment of a binary classification system that predicts diabetes risk from clinical measurements. It is intended for technical and non-technical reviewers alike — engineers can evaluate methodology and rigor, while business stakeholders can evaluate impact and applicability.

2.2 Problem Statement

Clinical diagnosis of diabetes typically requires laboratory testing (e.g., fasting glucose or HbA1c panels), which is accurate but resource-intensive and not something every patient receives proactively. A model that estimates risk from routinely collected measurements — glucose, BMI, blood pressure, age, pregnancy history, and similar — could act as a low-cost pre-screening layer, helping clinics prioritize who should be tested further.

2.3 Business Background

- Who experiences the problem: primary care providers and public health screening programs operating under time and resource constraints; patients who go undiagnosed until symptoms are advanced.
- Current limitations: lab-based screening at scale is expensive; symptom-based triage alone misses early-stage/asymptomatic cases.
- Existing solutions: risk-scoring questionnaires (e.g., ADA risk test) exist but are typically static rule-based scores rather than data-driven models.
- Expected value after solving it: a statistically grounded, easily-deployed pre-screening signal that can be embedded in a simple web form — no lab equipment required.

2.4 Objectives

Primary objective

Build a binary classifier that predicts diabetes status (Outcome = 0/1) from 8 clinical features, with performance and validation rigor suitable for a public portfolio and technical interviews.

Secondary objectives

- Correctly identify and resolve a non-obvious data quality issue (zero-encoded missing values).
- Compare multiple model families rather than relying on a single algorithm.
- Deploy the final model as a live, interactive application rather than leaving it in a notebook.

Success criteria / KPIs

KPI Type	Metric	Target / Result
Technical	AUC-ROC	$\geq 0.80 \rightarrow$ achieved 0.947
Technical	Cross-validated, not single-split, evaluation	Achieved (5-fold CV)
Business	Model usable without lab equipment	Achieved — 8 easily-measured inputs

Business	Publicly accessible demo	Achieved — Streamlit Cloud deployment
----------	--------------------------	---------------------------------------

2.5 Dataset

Attribute	Detail
Name	Pima Indians Diabetes Dataset
Source	National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK); distributed via Kaggle / UCI ML Repository
Rows	768 patient records
Columns	8 clinical features + 1 binary target (Outcome)
Population	Female patients of Pima Indian heritage, age 21+
Target variable	Outcome — 0 (no diabetes) / 1 (diabetes)
Class balance	500 negative (65.1%) / 268 positive (34.9%) — imbalanced
Duplicate records	[Information Needed]
Licensing / usage terms	[Information Needed]

Feature descriptions

Feature	Description	Missing-as-Zero?
Pregnancies	Number of times pregnant	0 is a valid value
Glucose	Plasma glucose concentration (2-hr oral glucose tolerance test)	Yes — 0.65% of rows
BloodPressure	Diastolic blood pressure (mm Hg)	Yes — 4.56% of rows
SkinThickness	Triceps skinfold thickness (mm)	Yes — 29.6% of rows
Insulin	2-hour serum insulin (mu U/ml)	Yes — 48.7% of rows
BMI	Body mass index	Yes — 1.43% of rows
DiabetesPedigreeFunction	Function scoring likelihood of diabetes based on family history	0 is a valid value
Age	Age in years	0 is a valid value

Data quality, bias, and ethical considerations

- Data quality: five clinical columns contain biologically impossible zero values that standard null-checks (`isnull().sum()`) do not catch, because they are encoded as 0 rather than NaN.
- Bias: the dataset exclusively contains female patients of Pima Indian heritage — a model trained on it should not be assumed to generalize to other populations, sexes, or ethnicities without further validation.
- Ethical considerations: this is a pre-screening aid, not a diagnostic tool. False negatives (missed diabetics) carry real health consequences, so the model should never replace clinical testing.
- Limitations: sample size (768) is modest by modern ML standards, and Insulin in particular is unreliable due to the high missingness rate even after imputation.

2.6 Methodology

The pipeline follows a standard supervised-learning workflow, adapted to handle the dataset's specific data-quality issue:

1. Load and inspect the raw dataset (shape, types, summary statistics).
2. Detect zero-encoded missing values in the five affected clinical columns.
3. Impute missing values using the median, computed separately within each Outcome class (grouped median), rather than a single global median.
4. Engineer new features: BMI category bins, glucose risk tiers, and age groups.
5. Conduct exploratory data analysis: distributions, correlation heatmap, class-conditional comparisons.
6. Split data into train/test sets (stratified, to preserve class balance) and scale features with `StandardScaler`.
7. Train three candidate models: Logistic Regression, Random Forest, and XGBoost.
8. Validate each model using 5-fold cross-validation rather than a single train/test split.
9. Evaluate using accuracy, precision, recall, F1, and AUC-ROC, with emphasis on the diabetic class.
10. Select the best-performing model (XGBoost) and serialize it with pickle.
11. Build a Streamlit application around the serialized model and deploy it to Streamlit Cloud.

2.7 Data Cleaning

Missing value handling

Zero values in Glucose, BloodPressure, SkinThickness, Insulin, and BMI were replaced with NaN, then imputed using the median value computed within each Outcome class — a more faithful estimate than a single dataset-wide median, since glucose and BMI distributions differ meaningfully between diabetic and non-diabetic patients.

Outlier handling

Outliers were flagged (e.g., via IQR-based checks) and reviewed rather than removed wholesale, to avoid discarding valid physiological extremes that are especially common — and clinically relevant — in a diabetic population.

Encoding, scaling, and feature selection

- No categorical encoding was required beyond the engineered bins (BMI category, glucose tier, age group), which were themselves derived from continuous variables.

- StandardScaler was applied to numeric features prior to training all three models (tree-based models — Random Forest, XGBoost — do not require feature scaling).
- All 8 original features plus the engineered features were retained for model training; no formal feature-selection step (e.g., recursive feature elimination) was applied. [Information Needed]

2.8 Exploratory Data Analysis

Distributions

Feature	Distribution Shape	Observation
Glucose	Near-normal, slight right skew	Best single separator between classes
BMI	Near-normal	Centered around ~32 (overweight range)
Age	Right-skewed	Most patients aged 21–40
Insulin	Heavily right-skewed	48.7% zero-imputed — treat with caution
Pregnancies	Right-skewed	Mode = 1, max = 17
SkinThickness	Right-skewed with zero spike	29.6% zero-imputed
DiabetesPedigreeFunction	Right-skewed	Encodes family-history risk numerically
BloodPressure	Near-normal	Weakest predictor — a counter-intuitive finding

Correlation findings

- Glucose ↔ Outcome: strongest positive correlation (~0.47) — the single most predictive feature.
- BMI ↔ SkinThickness: ~0.65 — both act as proxies for body composition.
- Age ↔ Pregnancies: ~0.54 — older patients have typically had more pregnancies.
- Insulin ↔ SkinThickness: ~0.44 — moderate relationship, though Insulin's reliability is limited by missingness.
- BloodPressure ↔ Outcome: surprisingly weak — suggests hypertension alone is a poor stand-alone screening signal.

2.9 Feature Engineering

- BMI category: bucketed BMI into clinical bands (underweight / normal / overweight / obese) to capture non-linear risk thresholds.
- Glucose risk tier: bucketed glucose into risk tiers aligned with clinical screening thresholds (e.g., normal / prediabetic / diabetic range).
- Age group: bucketed age into ranges to capture life-stage risk patterns not fully linear in raw age.

2.10 Model Development

Model	Why chosen	Hyperparameters
-------	------------	-----------------

Logistic Regression	Fully interpretable coefficients; strong baseline for clinical use where explainability matters	max_iter=1000; [Information Needed] for regularization strength / solver
Random Forest	Captures non-linear feature interactions without manual encoding	[Information Needed]
XGBoost	Gradient boosting typically improves recall on imbalanced tabular data	[Information Needed]

Training and validation strategy

- Train/test split: 80/20, stratified by Outcome, random_state = 42 for reproducibility.
- Cross-validation: 5-fold, stratified, applied to all three candidate models to reduce dependence on any single lucky/unlucky split.
- Formal hyperparameter tuning (GridSearchCV / RandomizedSearchCV): [Information Needed] — flagged as a priority next step (see Section 2.14).

2.11 Evaluation

Because the dataset is imbalanced (65.1% / 34.9%), accuracy alone is a misleading metric — a model that always predicts "no diabetes" would score 65% while being clinically useless. Precision, recall, and AUC-ROC for the diabetic class are treated as the primary metrics.

Metric	Logistic Regression	Random Forest	XGBoost
Accuracy	70.78%	86.36%	88.31%
AUC-ROC	0.826	0.945	0.947
Precision (diabetic)	58.82%	81.13%	82.14%
Recall (diabetic)	55.56%	79.63%	85.19%
F1 (diabetic)	57.14%	80.38%	83.64%

XGBoost was ultimately selected as the reported/deployed model based on its combination of the highest accuracy (88.31%), highest AUC-ROC (0.947), and — critically for a health screening context — the highest recall (85.2%) among the three models tested. Random Forest was a close second (86.36% accuracy, 0.945 AUC-ROC, 79.6% recall), and Logistic Regression served as the interpretable baseline (70.78% accuracy, 0.826 AUC-ROC, 55.6% recall).

Explainability

- Model explainability: feature importance derived directly from the XGBoost model's built-in importance scores (see Section 2.12).
- SHAP / LIME analysis: [Information Needed] — listed as a future enhancement (Section 2.14).
- Confusion matrix, ROC curve, precision-recall curve: [Information Needed] — recommended additions to the notebook prior to publishing the technical report externally.

2.12 Results

Feature importance (XGBoost, ranked)

Rank	Feature	Importance	Clinical Meaning
1	Insulin	~46% — dominant	Despite the highest missingness pre-imputation, the single strongest predictor by a wide margin after group-median imputation
2	SkinThickness	~11%	Body-fat proxy — second-strongest signal
3	Glucose	~10.5%	High blood sugar — the classic diabetes marker, but not the dominant one in this model
4	Age	~10%	Risk increases with age
5	BMI	~6.5%	Obesity is a well-established risk factor
6	BloodPressure	~6%	Lower predictive value than clinical intuition would suggest
7	DiabetesPedigreeFunction	~5.5%	Family history matters, but contributes less than metabolic markers
8	Pregnancies	~5.5%	Linked to gestational diabetes history — weakest signal of the eight

2.13 Business Insights

- Insulin is the single most influential feature in the deployed model (~46% importance) — a genuinely counterintuitive finding worth flagging for clinical review, since Glucose is the biomarker most associated with diabetes in established medical literature but ranks third here.
- Patients with glucose > 140 mg/dL show a dramatically higher predicted risk in this model.
- BMI and family history compound risk — the model implicitly captures this even without an explicit interaction term.
- Blood pressure alone is a weak predictor, suggesting hypertension screening should be paired with glucose/BMI data rather than used as a stand-alone signal.
- Recall on the diabetic class reaches 85.19% in the deployed XGBoost model — a meaningful improvement over the Logistic Regression baseline's 55.56%. In a screening context this matters directly: XGBoost misses 8 of 54 diabetic patients in the test set versus 24 missed by the baseline. Continued improvement here remains valuable, since any missed case in a health screening context carries real cost.

2.14 Recommendations / Future Work

Priority	Recommendation
High	Add a precision-recall curve to complement the existing confusion matrices and ROC curve comparison

High	Address class imbalance explicitly via SMOTE or <code>class_weight='balanced'</code>
High	Continue monitoring model performance as more patient data becomes available, and consider recalibrating the risk threshold based on clinical feedback
Medium	Hyperparameter tuning via GridSearchCV / RandomizedSearchCV
Medium	Add SHAP values for per-prediction explainability
Medium	Surface model confidence scores in the Streamlit UI
Low	Automated retraining pipeline (e.g., GitHub Actions) if the dataset is ever refreshed
Low	Extend the portfolio with the bank churn prediction project as a second case study

2.15 Challenges

- Hidden missing data: `standard.isnull().sum()` checks reported zero missing values, masking the fact that five columns used 0 as a missing-value sentinel. Identifying this required domain reasoning (a live patient cannot have 0 glucose or 0 BMI), not just automated checks.
- Class imbalance: required deliberately choosing metrics (precision/recall/AUC-ROC) that wouldn't be flattered by a naive high-accuracy, low-usefulness model.
- Deployment: Streamlit Cloud initially failed to build due to pinned package versions in `requirements.txt`; resolved by switching to unpinned version ranges.
- Environment inconsistency: variable-naming drift across notebook cells and mismatched Python environments between the terminal and IDE required standardizing on Anaconda Prompt for reliable execution.

2.16 Conclusion

This project delivers a complete, defensible data science pipeline — from a subtle data-quality discovery through to a publicly deployed, interactive prediction tool. The modeling choices are honest about the dataset's limitations (imbalance, missingness, modest size) rather than overselling results, and the deployment closes the loop from analysis to a usable product.

2.17 Appendix

Glossary

Term	Definition
AUC-ROC	Area under the Receiver Operating Characteristic curve; measures a classifier's ability to distinguish classes across all thresholds
Precision	Of all cases predicted positive, the share that were actually positive
Recall (Sensitivity)	Of all actual positive cases, the share the model correctly identified
Stratified split	A train/test split that preserves the original class proportions in both subsets
Grouped median imputation	Filling missing values using the median computed separately within each target class, rather than the whole dataset

2.18 References

- Pima Indians Diabetes Dataset — National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK), distributed via Kaggle and the UCI Machine Learning Repository.
- International Diabetes Federation (IDF), 2021 global diabetes prevalence estimate (537 million adults).

3. Portfolio Case Study

Project Overview

Diabetes Risk Prediction is an end-to-end machine learning project that turns a public clinical dataset into a live, interactive pre-screening tool. It covers the full lifecycle recruiters look for: messy real-world data, honest handling of a subtle data-quality trap, rigorous model comparison, and a deployed product — not just a notebook.

The Problem

Could a model flag diabetes risk using only 8 easily-measured clinical values, cheaply enough to serve as a pre-screening layer ahead of full lab testing?

The Solution

An XGBoost classifier (compared against Random Forest and a Logistic Regression baseline) trained on the Pima Indians Diabetes dataset, achieving 88.31% accuracy and 0.947 AUC-ROC, deployed as a public Streamlit web app where a user can enter their own values and get an instant risk read.

Approach

12. Diagnosed a hidden data-quality issue: five clinical columns used 0 to represent missing data, invisible to standard null checks.
13. Cleaned the data with grouped-median imputation and engineered clinically meaningful features (BMI category, glucose tier, age group).
14. Trained and cross-validated three model families rather than committing to the first one that worked.
15. Evaluated with metrics appropriate for imbalanced classification, not raw accuracy.
16. Shipped the winning model as a live, public web application.

Tools Used

Python · Pandas · NumPy · Scikit-learn · XGBoost · Matplotlib · Seaborn · Streamlit · Streamlit Cloud · Git/GitHub · Jupyter Notebook · Anaconda

Results

- 88.31% test accuracy against a 65.1% naive baseline
- 0.947 AUC-ROC
- Three models formally compared via 5-fold cross-validation
- Live, publicly accessible demo application

Impact

Demonstrates the ability to take a project from raw, imperfect data all the way to a shipped product — the same arc expected of a junior data scientist or ML engineer on a real team.

Lessons Learned

- `isnull().sum()` is not a substitute for domain reasoning about what 'missing' looks like in a specific dataset.
- Accuracy is a dangerous default metric on imbalanced data.

- Deployment friction (like pinned dependency versions breaking a cloud build) is a normal, expected part of shipping — not a sign something is fundamentally wrong.

Screenshots to include

- Streamlit app home screen with the prediction form
- A sample prediction result screen
- The correlation heatmap from the EDA notebook
- The model comparison / metrics output

Links

Resource	Link
Live Demo	[Insert Streamlit URL]
GitHub Repository	github.com/yemifatodu/diabetes-prediction-app
Portfolio	yemifatodu.online

4. Portfolio Website Page Copy

Ready-to-paste copy for a dedicated project page at yemifatodu.online, organized by section.

Hero

Diabetes Risk Prediction

A machine learning pre-screening tool that flags diabetes risk from 8 routine clinical measurements — trained on 768 patient records, validated with 5-fold cross-validation, and deployed as a live web app.

[[View Live Demo](#)] [[View on GitHub](#)]

Overview

This project takes a public clinical dataset from raw CSV to a deployed, interactive prediction tool — covering data cleaning, exploratory analysis, feature engineering, model comparison, and public deployment.

Problem

Diabetes affects over 537 million adults globally. Lab-based screening is accurate but resource-intensive. This project asks whether a lightweight model, built on easily-collected values, can serve as a useful pre-screening signal.

Objectives

- Build an interpretable classifier suitable for a clinical pre-screening context
- Handle a real, non-obvious data-quality problem rigorously
- Compare multiple model families instead of settling for the first one
- Ship a working, public demo — not just a notebook

Technologies

Python, Pandas, NumPy, Scikit-learn, XGBoost, Matplotlib, Seaborn, Streamlit, Streamlit Cloud, Git/GitHub, Jupyter Notebook.

Architecture & Workflow

Raw CSV → Cleaning (zero-as-missing detection, grouped-median imputation) → Feature Engineering → EDA → Model Training (Logistic Regression / Random Forest / XGBoost, 5-fold CV) → Model Selection → Pickle Serialization → Streamlit App → Streamlit Cloud Deployment.

Exploratory Data Analysis

Correlation analysis identified Glucose ($r \approx 0.47$) as the strongest single predictor of diabetes outcome, with BMI and family history (DiabetesPedigreeFunction) as secondary drivers. Blood pressure, counter-intuitively, was the weakest predictor in the set.

Model

XGBoost was selected as the deployed model for its combination of strong AUC-ROC (0.947) and the highest recall of the three models tested — an important property in a clinical-adjacent application where a missed positive case carries real cost.

Results

88.31% test accuracy (vs. a 65.1% naive baseline) and 0.947 AUC-ROC, validated via 5-fold cross-validation rather than a single train/test split.

Business Insights

Insulin is the model's single strongest predictor (~46% importance), ahead of Glucose — a genuinely counterintuitive finding worth flagging for clinical review, since Glucose is the biomarker most associated with diabetes in established medical literature. A pre-screening tool built around the full feature set, not Glucose alone, could help prioritize patients for further lab testing.

Deployment

The trained model is serialized with pickle, wrapped in a Streamlit interface, and deployed publicly via Streamlit Cloud — with the source hosted on GitHub for full transparency.

Lessons Learned

The most valuable finding wasn't a modeling technique — it was catching that five clinical columns used 0 to silently represent missing data, a trap that standard missing-value checks do not catch.

Future Work

Add SHAP-based explainability, address class imbalance directly with SMOTE, and expand the app to show model confidence alongside each prediction.

Call to Action

Interested in the details?

Explore the full code and notebooks on GitHub, or try the live prediction tool yourself.

[[View Live Demo](#)] [[View on GitHub](#)] [[Contact Me](#)]

5. Interview Preparation

5.1 Technical Interview Questions

17. Walk me through your data pipeline from raw CSV to deployed model.
18. How did you detect the zero-encoded missing values, and why didn't standard null checks catch them?
19. Why did you impute missing values grouped by Outcome instead of using a single global median?
20. How did you avoid data leakage between your train and test sets?
21. Why was feature scaling necessary for Logistic Regression but not for your tree-based models?

5.2 Machine Learning Interview Questions

22. Why is accuracy a misleading metric on this dataset, and what did you use instead?
23. Explain what AUC-ROC of 0.947 actually tells you about the model.
24. Why did you choose 5-fold cross-validation instead of a single 80/20 split?
25. If you had to improve recall on the diabetic class, what would you try first?
26. How would you explain a single prediction from your Logistic Regression model to a non-technical stakeholder?

5.3 SQL Questions

This project's data was loaded from CSV rather than a SQL database, so project-specific SQL examples are [Information Needed]. General SQL prep questions relevant to a data science role:

27. Write a query to find the top 5 patients by glucose level within each Outcome group (window functions).
28. How would you calculate the percentage of missing (zero-encoded) values per column directly in SQL?
29. Explain the difference between INNER JOIN and LEFT JOIN with an example relevant to patient/visit data.

5.4 Python Questions

30. How did you structure your imputation logic — walk through the pandas code.
31. What's the difference between `.fillna()` with a scalar vs. a `groupby().transform()` approach, and which did you use?
32. How did you serialize your final model, and what are the security/versioning risks of using pickle?
33. Show how you'd load your saved model inside the Streamlit app.

5.5 Business/Scenario Questions

34. If this model were used in a real clinic, what's the cost of a false negative vs. a false positive, and how should that shape your threshold choice?
35. How would you explain to a hospital administrator why 88.31% accuracy is actually a strong result here?
36. The dataset only contains one demographic group — how would you communicate that limitation to a client considering broader deployment?

37. Streamlit Cloud deployment failed initially — walk me through how you diagnosed and fixed it.

5.6 STAR-Format Answers

Example 1 — Problem-Solving

Situation: Initial null-checks on the dataset reported zero missing values.

Task: Ensure the model wasn't being trained on biologically invalid data.

Action: Investigated column-level minimums and domain plausibility, discovered zero-encoded missing values in five clinical columns, and replaced them with grouped-median imputation.

Result: Cleaner, clinically valid training data and a more defensible model — and a strong story about attention to data quality.

Example 2 — Shipping Under Constraints

Situation: The Streamlit Cloud deployment failed to build.

Task: Get the app live without destabilizing the local development environment.

Action: Diagnosed the failure as pinned package versions in requirements.txt incompatible with the cloud build environment, and switched to unpinned version ranges.

Result: Successful public deployment, and a reusable lesson about dependency management across environments.

6. Recruiter Summary

Yemi Fatodu is a data science student/intern who took a public healthcare dataset through a complete, production-minded ML lifecycle — from a subtle data-quality discovery to a live, publicly deployed application.

Business value

Built a working proof-of-concept for a low-cost diabetes pre-screening tool with clear, honest performance framing — 88.31% accuracy against a 65.1% baseline, 0.947 AUC-ROC — and a plain-language explanation of why that matters clinically.

Technical skills demonstrated

- Data cleaning & quality diagnosis (Pandas, NumPy)
- Exploratory data analysis & visualization (Matplotlib, Seaborn)
- Classical ML and gradient boosting (Scikit-learn, XGBoost)
- Model validation (stratified k-fold cross-validation)
- Application development & deployment (Streamlit, Streamlit Cloud, Git/GitHub)

Problem-solving

Independently diagnosed a hidden data-quality issue that standard checks missed, and resolved a real cloud-deployment failure caused by dependency pinning.

Communication

Documented findings, blockers, and resolutions throughout the project and produced portfolio-ready technical and non-technical write-ups of the same work.

Results

Result	Detail
Model performance	88.31% accuracy / 0.947 AUC-ROC, 5-fold cross-validated
Models compared	Logistic Regression, Random Forest, XGBoost
Deployment	Live public Streamlit application + GitHub repository

7. Resume Project Entry

Diabetes Risk Prediction — Personal Project

[Information Needed] (dates)

- Built and deployed a diabetes risk classifier on 768 patient records, achieving 88.31% accuracy and 0.947 AUC-ROC, validated with 5-fold cross-validation across 3 model architectures (Logistic Regression, Random Forest, XGBoost).
- Identified and resolved a hidden data-quality issue affecting 5 of 8 clinical features (zero-encoded missing values undetected by standard null checks), improving data integrity prior to modeling.
- Engineered 3 new clinically-informed features (BMI category, glucose risk tier, age group) to improve model signal.
- Deployed a public-facing prediction application using Streamlit and Streamlit Cloud, resolving a dependency-pinning build failure to ship a live demo.
- Version-controlled the full pipeline on GitHub with a documented, reproducible workflow from raw data to deployed model.

8. Portfolio SEO Content

Field	Content
Meta Title	Diabetes Risk Prediction ML Pre-Screening Tool — Yemi Fatodu
Meta Description	A machine learning project predicting diabetes risk from clinical data — data cleaning, model comparison, and a live Streamlit demo. See the full case study.
Keywords	diabetes prediction machine learning, Pima Indians dataset, healthcare data science project, Streamlit deployment, logistic regression classification, data science portfolio
Open Graph Description	How I built and deployed a diabetes risk pre-screening model — from a hidden data-quality bug to a live web app. Explore the full technical write-up.
Project Tags	Machine Learning · Healthcare · Python · Scikit-learn · Streamlit · Classification · Data Cleaning · Portfolio Project

Prepared as a consolidated reference package. Sections marked [Information Needed] should be filled in directly from your notebook outputs (Random Forest / XGBoost metrics, hyperparameters, confusion matrix) and personal details (project duration) before external publication.